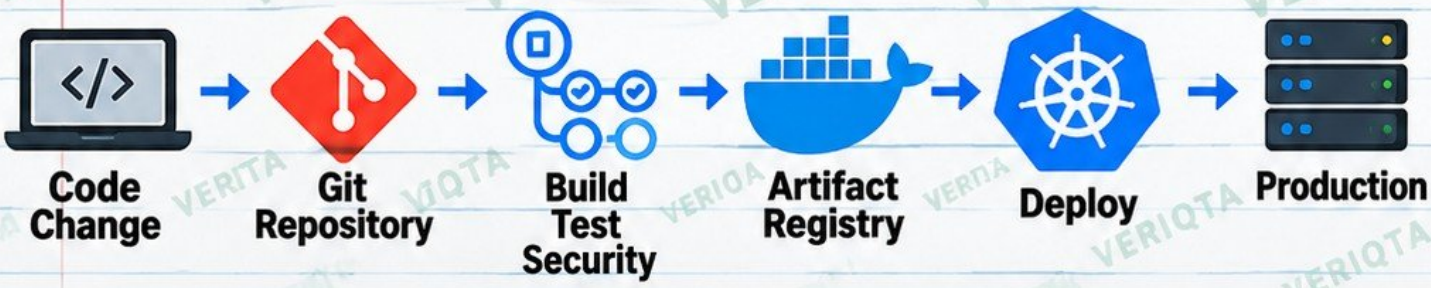
VERIQTA

15 CI/CD Mistakes That Break Production (and How to Avoid Them)



COMMON MISTAKES

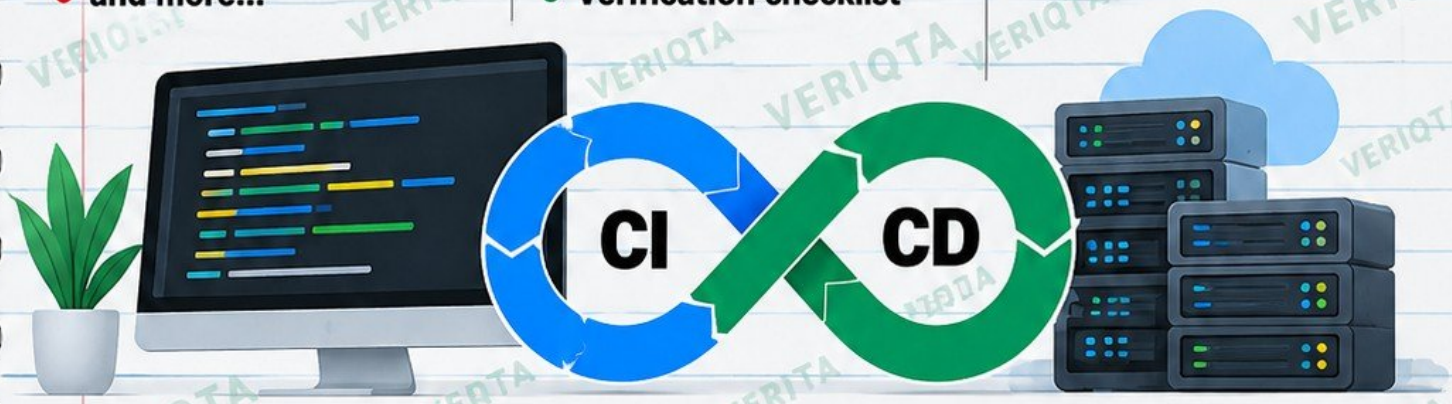
- Broken deployments
- Security risks
- Configuration errors
- Database issues
- Pipeline misconfigurations
- Environment drift
- Insufficient testing
- and more...

PRACTICAL SOLUTIONS

- Real examples
- Step-by-step fixes
- Best practices
- Production guidance
- Security controls
- Troubleshooting tips
- Deployment strategies
- Verification checklist

REAL DEVOPS SCENARIOS

- Learn from real failures
- Understand the impact
- How to prevent them
- How to detect them early
- How to recover safely
- Build a production-safe pipeline



@veriqta

01. How a CI/CD Pipeline Can Break Production

Continuous Integration (CI)

- Automatically builds and tests code. Integrates changes often (usually many times a day).
- Helps find issues early before deployment.

Continuous Delivery (CD)

- Automates the release process after successful CI.
- Produces a deployable artifact ready for production.
- Requires a manual approval step to deploy to production.

Continuous Deployment

- Automatically deploys changes to production after all checks pass.
- No manual approval (fully automated).
- Used when you have strong tests, monitoring and safe deployment practices.

How Code Moves Through a CI/CD Pipeline



Developer
Writes code and pushes changes

Git Repository
Stores code and triggers the pipeline

CI Checks
Builds the code, runs tests and security scans

Artifact Registry
Stores the built artifact (container image or package)

Deployment
Deploys the approved artifact

Production
Runs for real users

Monitoring
Tracks health, performance and errors

Why a Successful Pipeline Does Not Always Mean a Healthy Application

- The pipeline only checks what you configured. It can miss real application issues.
- Tests may pass, but the change can still break in a real environment.
- Configuration, secrets or environment differences may cause failures after deployment.
- Database changes, third-party dependencies or runtime behavior may not be fully tested.
- The pipeline can finish successfully while users experience errors in production.

Four Ways a CI/CD Pipeline Can Break Production

1. Incorrect Changes

- Code contains bugs.
- Feature not fully tested.
- Breaking changes to APIs or database.
- Wrong configuration values.

2. Missing Safeguards

- No meaningful tests.
- Security checks skipped.
- Approvals not required.
- Weak review process.
- Secrets exposed.

3. Unsafe Deployments

- Deploying all instances at once.
- Wrong environment.
- Using mutable image tags (e.g. latest).
- Misconfigured health checks.

4. Failed Recovery

- No rollback plan.
- Previous version not available.
- Database changes not reversible.
- Lack of monitoring to detect and respond early.

Production Insight | A pipeline can finish successfully while users experience errors. Always verify application behavior in production after deployment.

02. The Anatomy of a Production Deployment Failure

Realistic Scenario

A new version of an application is deployed to production. The pipeline completes successfully. A few minutes later, users start reporting errors and the monitoring system shows a sharp increase in **HTTP 500 errors**.

What Happened?

- The deployment completed without errors.
- The application started receiving traffic.
- Error rates increased suddenly.
- Users experienced failures.
- Engineers need to investigate and restore service.

Deployment Failure vs Application Failure

Deployment Failure

- The deployment process fails.
- New version is not fully deployed.
- Examples: image pull error, configuration error, permission denied.
- The application may still be running the previous version.

Application Failure after Deployment

- The deployment succeeds.
- The new version is running.
- The application has runtime issues (bugs, configuration, database, dependencies).
- Users experience errors even thongh the pipeline was successful.

Why Evidence Matters

- Deployment timestamp helps you connect the issue to the release.
- Logs show what the application is doing and any error messages.
- Metrics show when the error started and how many users are affected.
- Recent changes (code, configuration, infrastructure, database) help identify the most likely cause.

From Deployment to Incident



Code Change
Developer pushes new code.

Git Repository
Triggers the pipeline.

Successful Deployment
Pipeline completes and deploys new version.

HTTP 500 Errors
Users start experiencing errors.

Alert
Monitoring detects the increase in errors.

Investigation
Engineers check logs, metrics and recent changes.

Recovery
Fix the issue and restore normal service.

Investigation Process

1. **Detect** — Identify that something is wrong (alert, user report, monitoring).
2. **Assess Impact** — Understand which services and users are affected.
3. **Mitigate** — Take immediate action to reduce impact (rollback, scale, traffic shift, fix configuration).
4. **Investigate** — Find the root cause using logs, metrics and recent changes.
5. **Verify** — Confirm the application is healthy and errors have stopped.

Key Questions to Ask

- When did the errors start?
- What changed recently?
- Was there a recent deployment?
- Are the new application instances running correctly?
- Do the logs show any errors?
- Are database or external services working?
- Is the issue related to configuration or environment changes?

Practice

? A previously healthy application starts returning HTTP 500 errors immediately after a deployment. Which evidence would you check first?

- **A** Application logs
- **B** Deployment timestamp
- **C** System CPU usage
- **D** Container image size

Mistake 01: Deploying Without Meaningful Automated Tests

What Goes Wrong?

- The pipeline builds successfully and deploys code that contains application defects.
- The code compiles, but it does not work correctly in real environments.
- Users experience errors after deployment.

Build vs Testing Application Behavior

Build Successfully

- Code compiles.
- Dependencies are installed.
- Artifact is created.
- Does not check if the application works correctly.

Testing Application Behavior

- Checks if the application actually works.
- Validates real use cases.
- Finds logic errors, integration issues and breaking changes.
- Helps prevent production incidents.

Types of Automated Tests

Unit Tests



- Test small pieces of code (functions or classes).
- Fast and isolated.
- Catch basic logic errors early.

Integration Tests



- Test how components work together (e.g. application + database).
- Find issues that unit tests miss.
- Use real or realistic dependencies.

Contract Tests



- Validate the format of requests and responses between services.
- Ensure one service does not break another service.
- Important for microservices.

Smoke Tests

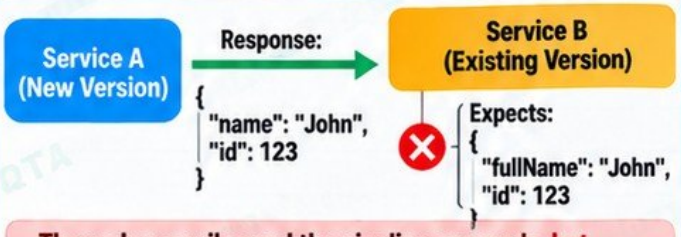


- Run basic checks after deployment.
- Confirm the application is up and responding.
- Help detect major issues quickly.

Why Mocked Dependencies Are Not Enough

- Mocks are useful for unit tests, but they cannot fully simulate real systems.
- Real integration tests are needed to catch issues like:
  - Database connection problems
  - Authentication and permission issues
  - External API changes
  - Network or timeout issues
  - Data format mismatches

Example: API Change That Breaks Another Service



The code compiles and the pipeline succeeds, but Service B fails because the response format changed. This should be caught by integration or contract tests.

How to Make Tests Required in the Pipeline

- Run relevant unit, integration, contract and smoke tests.
- Make these tests blocking (pipeline fails if they fail).
- Do not allow deployment if required tests are skipped.
- Treat test failures as release blockers.
- Review and fix flaky tests instead of ignoring them.

Prevention

- Write meaningful tests for real application behavior.
- Include integration and contract tests for critical services.
- Run smoke tests after deployment.
- Block releases when required tests fail.
- Continuously improve test coverage based on past incidents.

Junior Engineer Tip | A green build confirms the build step succeeded, not that the software is correct.

Mistake 02: Ignoring Failed Tests and Quality Gates

What Goes Wrong?

- Pipelines are configured to continue even when important tests or security checks fail.
- Failed steps do not stop the deployment.
- Defects, security issues or breaking changes reach production.

Required Checks and Exit Codes

- Each test or scan returns an exit code.
- 0 means success.
- Any non-zero value (e.g. 1) means failure.
- In a properly configured pipeline, a failed step stops the pipeline and prevents deployment.
- If failure is ignored, the pipeline continues and later steps (including deployment) can still run.

How Failed Tests Can Be Ignored in GitHub Actions

Incorrect (hides failure)

```
- name: Run tests
  run: npm test
  continue-on-error: true
```

- The job continues even if tests fail.
- Deployment can still run.

Correct (blocks pipeline)

```
- name: Run tests
  run: npm test
```

- If tests fail, the job and pipeline stop.
- Later steps (e.g. deployment) will not run.

Optional vs Mandatory Checks

Optional Informational Checks

- Code formatting.
- Linting.
- Non-blocking scans.
- Performance checks.
- Documentation checks.

Useful for feedback, but not required to block a release.

Mandatory Release Gates

- Unit tests.
- Integration tests.
- Security scans.
- Contract tests.
- Build and artifact creation.
- Deployment approvals.

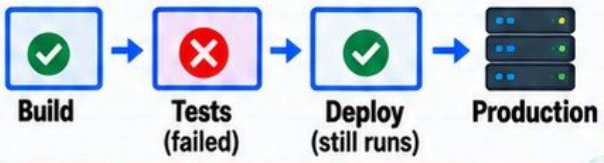
If these fail, the pipeline must stop and not deploy.

Why Retrying a Flaky Test Can Hide Defects



- Flaky tests sometimes fail due to real issues (e.g. timing, race conditions, environment problems).
- Constantly retrying without investigating can hide real defects.
- A flaky test today can become a production incident tomorrow.

Example: A Failed Test That Still Deploys



The test step failed, but because the pipeline was configured to continue, the deployment still ran.

Troubleshooting Steps

1. Check the pipeline logs for failed steps.
2. Look at the exit status of the failed job.
3. Find any continue-on-error, allow_failure or similar settings.
4. Check which steps are marked as required in branch protection rules.
5. Verify if deployment steps depend on successful tests.

Prevention

- Make critical tests and security checks blocking.
- Remove continue-on-error for important steps.
- Investigate and fix flaky tests instead of simply retrying.
- Use required status checks in your repository settings.
- Ensure deployment jobs only run when all required checks pass.

Junior Engineer Tip

A green pipeline does not always mean the application is correct. It only means the configured steps completed successfully.

Mistake 03: Building Directly From Unreviewed Code

What Goes Wrong?

- Code is pushed directly to a production or release branch without proper review.
- Unsafe or incorrect changes reach the deployment workflow.
- Developers may accidentally or intentionally modify the pipeline, deployment scripts or infrastructure code.
- This can introduce bugs, security risks or breaking changes in production.

Key Concepts

- **Protected Branches**
  Restrict who can push, require pull requests and enforce required checks.
- **Pull Requests (PR)**
  A controlled way to propose changes, review them and discuss risks.
- **Code Reviews**
  Team members review code for correctness, security and impact.
- **Required Status Checks**
  CI tests, security scans and other checks must pass before merging.

How the Normal Flow Works



Developer
creates a branch and pushes code



Pull Request
Code is reviewed and discussed



Required Checks
Tests, security scans and other checks pass



Merge to Protected Branch
Only after approval and all checks pass



Deployment Pipeline
Runs with reviewed and validated code

Unreviewed Workflow Changes Can Break Production

Example: A workflow change accidentally removes a security scan.

Original Workflow (with security scan)

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: npm test
      - name: Run security scan
        run: npm audit
```

Modified Workflow (scan removed)

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Run tests
        run: npm test
  # Security scan removed
```



This change can go to production if it is not properly reviewed, removing an important security check.

CODEOWNERS and Approval Requirements

- Use a **CODEOWNERS** file to require specific people to review sensitive changes.
- Require approvals for changes to:
  - CI/CD workflows (e.g. .github/workflows/)
  - Deployment scripts
  - Infrastructure as Code (Terraform, etc.)
  - Environment configuration
  - Secrets and access permissions

Example CODEOWNERS entry:

```
# Require approval for workflow changes
.github/workflows/ @devops-team
```

Why Branch Protection Alone Is Not Enough

- Privileged users (e.g. repository admins) may be able to bypass the rules.
- Misconfigured settings can allow direct pushes.
- Insider threats or compromised accounts can still make unsafe changes.
- Branch protection must be combined with required reviews, status checks, restricted permissions and audit monitoring.

Prevention

- Require pull requests for all changes.
- Enforce code reviews and required status checks.
- Protect release and main branches.
- Use CODEOWNERS for sensitive files.
- Restrict bypass permissions to a small number of trusted users.
- Review and approve all pipeline and deployment changes.
- Monitor and audit changes to workflows and infrastructure code.



Production Awareness

Pipeline configuration is production-critical code.
A small change in a workflow file can remove security checks, deploy untested code, or give unintended access to production environments.



Junior Engineer Tip

Always review changes to CI/CD workflows, deployment scripts and infrastructure files with the same care as application code.

Mistake 04: Exposing Secrets Through CI/CD Pipelines

What Goes Wrong?

- Credentials are leaked through repositories, build logs, artifacts or compromised pipeline jobs.
- Secrets are hardcoded in code or configuration files.
- Excessive token permissions give attackers full access.
- Untrusted pull request workflows can expose secrets.
- Stolen credentials can be used to access cloud resources and deploy malicious changes.

Common Types of Secrets

Access Tokens
GitHub tokens, API tokens and service tokens.

Cloud Credentials
AWS, Azure, GCP keys and roles.

Deployment Identities
Service accounts and workload identities.

Database Credentials
Usernames, passwords and connection strings.

Third-Party API Keys
Keys for external services and tools.

Where Secrets Can Be Exposed

- Source code repositories (hardcoded secrets).
- Build logs (printed by mistake).
- Build artifacts (files or images).
- Untrusted pull request workflows.
- Excessive permissions on tokens or roles.
- Compromised runner or pipeline job.

Example: A Pipeline Prints a Deployment Token

```yaml
- name: Debug deployment
  run: |
    echo "Deploying to production ..."
    echo "Token: $DEPLOY_TOKEN"
  - ./deploy.sh
```

Output in CI/CD Logs

```
Deploying to production ...
Token: ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

The token is exposed in the logs and can be used by anyone with access to the build logs.

Risks of Exposed Secrets

- Unauthorized access to cloud resources.
- Malicious deployments.
- Data theft or modification.
- Full account takeover.
- Persistent access if credentials are not rotated.
- Reputational and financial damage.

Safer Approach: OIDC Federation



Uses short-lived credentials. No long-lived secrets stored in the repository.

Hardcoded Secret vs Using a Secret

Hardcoded Secret (Unsafe)

```
api_key = "AKIAoxxxxxex"
db_password = "mypassword"
```

- Stored in code.
- Can be leaked publicly.
- Hard to rotate.

Using Pipeline Secret (Safer)

```
api_key = os.environ["API_KEY"]
db_password = os.environ["DB_PASSWOORD"]
```

- Stored in secret manager.
- Not exposed in code.
- Can be rotated easily.

How to Protect Secrets

- Use managed secrets (GitHub Secrets, cloud secret managers).
- Use least-privilege permissions.
- Use short-lived credentials and OIDC.
- Do not expose secrets in logs or artifacts.
- Protect production environments.
- Restrict access to secrets and runners.

Secret Rotation and Revocation

1. Revoke exposed credentials immediately.
2. Rotate to new credentials.
3. Audit recent access.
4. Update configurations.
5. Monitor for suspicious activity.

Security Reminder | Masked log output does not guarantee a secret cannot be exposed through other channels. Always assume secrets can be leaked.

Junior Engineer Tip | Never print secrets in logs. Use managed secrets and short-lived credentials. Limit permissions to only what is required.

Mistake 05: Using Mutable Image Tags in Production

What Goes Wrong?

- Engineers deploy an image using a tag such as latest, but the tag later points to different image content.
- Different environments can unknowingly run different images under the same tag.
- This breaks consistency, traceability and can introduce unexpected bugs or security issues.

Key Concepts

Image Tags
Human-readable labels (e.g. latest, v1.2.0).

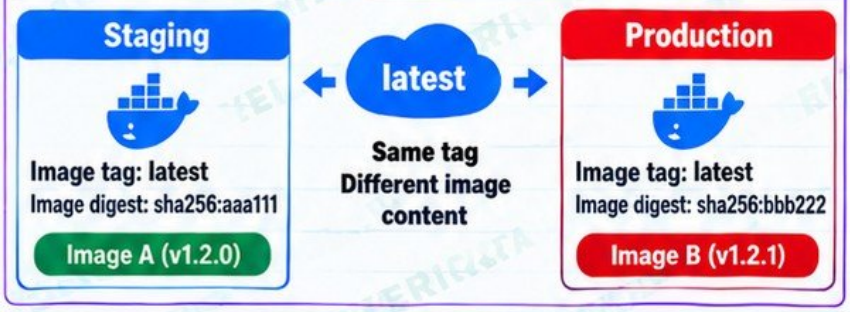
Image Digests
Immutable ID (e.g. sha256:...).

Artifact Immutability
Once built, the image should not change.

Why It Happens

- Mutable tags (e.g. latest) can point to new images over time.
- Rebuilding the same source code can create a different image (e.g. new base image, dependencies, build timestamp).
- Lack of traceability between code, image and deployment.

How Different Environments Can Get Different Images



Why Rebuilding Can Change the Image

- Base image may have new packages.
- Build time and environment variables may differ.
- Dependencies may be updated.
- The same code can produce a different image even without code changes.

Example: Staging/Production Mismatch

Staging tests one container image, but production pulls another image using the same mutable tag (e.g. latest). The application behaves differently in production.

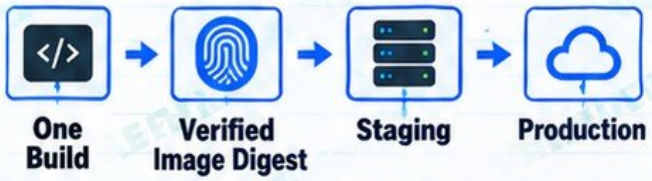
Real-World Impact

- Hard-to-debug issues.
- Inconsistent behavior across environments.
- Security and compliance risks.

How to Use Image Digests and Gain Traceability

- Build the image once and record the digest.
- Use the immutable digest in environments.
- Track the image tags for deployment records.
- Link the digest to the source code in the build pipeline.
- This guarantees the same exact image is deployed everywhere.

The Deployment Flow (Recommended)



Prevention

- Build once, promote the same artifact.
- Deploy by immutable digest where supported.
- Avoid using mutable tags in production.
- Keep proper traceability between code, image and deployment.

Junior Engineer Tip

Don't trust the label.
A digest is the real identity of your image.

Mistake 06: Skipping Dependency and Artifact Security Checks

What Goes Wrong?

- A release contains vulnerable dependencies, compromised packages, or untrusted build artifacts.

Key Concepts

Dependency Scanning
Checks libraries and packages for known vulnerabilities.

Container Scanning
Checks container images for vulnerabilities and malware.

Supply-Chain Risks
Protects against compromised packages, fake artifacts and untrusted sources.

Finding vs Real Risk

Vulnerability Finding
- Known issue in a dependency or package.
- May or may not be exploitable in your environment.

Actively Exploitable Risk
- Can be used in production.
- Requires immediate action if reachable and exploitable.

Supply-Chain Security

SBOM (Software Bill of Materials)
Lists all components in your build.

Artifact Provenance
Shows where the artifact came from and how it was built.

Signature Verification
Confirms the artifact is authentic and not tampered with.

Why Scanning Only at Deployment Is Too Late

- Vulnerable dependencies can be exploited before deployment (e.g. in development or build environments).
- Compromised packages can spread through caches and registries.
- Malicious artifacts can run in your build or test environments and steal secrets.

Common Risks

- Vulnerable dependencies (e.g. outdated libraries).
- Compromised packages (e.g. typosquatting).
- Untrusted artifacts (e.g. from unknown sources).
- Tampered images (e.g. modified containers).

Example: Vulnerable Dependency in a Container Image

```
$ trivy image myapp:latest
...
  libxml2 2.9.10 -> 2.9.14    HIGH
    CVE-2023-xxxx  (RCE)
  ...
```

- The scan finds a known vulnerability in a dependency.
- It is not automatically exploitable, but it must be evaluated and fixed if it can be reached in production.

Prevention

- Scan dependencies and artifacts in the pipeline.
- Evaluate severity and exploitability.
- Verify artifact origins and signatures.
- Establish a documented exception process for valid risks.

Security Checks in the Build Pipeline

Dependencies → Scan for vulnerable packages

Container Image → Scan for OS and package issues

SBOM → Generate and store SBOM

Provenance → Verify build origin and integrity

Signature → Verify artifact signature

Deploy → Only deploy if all checks pass

Junior Engineer Tip

A vulnerability scanner produces evidence for decisions. It does not automatically prove an application is secure.

Mistake 07: Allowing Development and Production Environments to Drift

What Goes Wrong?

- Code behaves correctly in testing but fails in production due to different configuration, infrastructure or dependencies.
- Environment differences cause unexpected errors and outages.
- Missing permissions, network rules or service endpoints can break the application in production.
- Small configuration differences can create big problems.

What Is Environment Drift?

Environment drift happens when development, testing and production environments are not consistent.

Drift Can Include Differences In:

- Application versions and packages
- Environment variables and settings
- Infrastructure and instance types
- Permissions and access policies
- Network configuration and firewalls
- External service dependencies (e.g. databases, APIs, queues)

Why It Happens

- Manual configuration changes.
- Different infrastructure between environments.
- Untracked environment variables and secrets.
- Different permissions and access controls.
- Network and firewall differences.
- Lack of infrastructure as code and configuration management.

Example: Works in Staging but Fails in Production

Staging (Works)

- Application connects to database successfully.
- Correct permissions.
- Required environment variables are set.
- All services accessible.

Application runs correctly in staging.

Production (Fails)

- Application cannot connect to database.
- Workload identity lacks required permission.
- Different network rules.
- Missing environment variable or different configuration.

Application fails in production with connection errors.

Key Insight

- Complete environment equality is not always possible.
- Critical application behavior should be tested in an environment that closely resembles production.
- Differences should be intentional, well documented and regularly reviewed.
- Assume that configuration, permissions and infrastructure can change independently across environments.

Using Infrastructure as Code

- Define infrastructure using code (e.g. Terraform, CloudFormation, ARM).
- Keep environment configuration in version control.
- Review and track changes through pull requests.

Detecting and Managing Drift

- Regularly compare configurations between environments.
- Use tools to detect infrastructure drift.
- Document approved differences.
- Review changes and update tests when environments change.

Prevention

- Version configuration and infrastructure.
- Use infrastructure as code.
- Review and track changes.
- Test with production-relevant dependencies and permissions.
- Monitor for configuration drift.
- Validate critical integrations in a staging environment that closely matches production.

Junior Engineer Tip

A working application in staging does not guarantee it will work in production. Always check configuration, permissions, network access and external dependencies.

Mistake 08: Deploying Database Changes Without Compatibility Planning

What Goes Wrong?

- A database schema change breaks application versions that are still running.
- Removing or modifying a column can cause runtime errors.
- A rolling deployment may run both old and new application versions at the same time.
- Data loss or service outage can occur if changes are not backward compatible.

Key Concepts

Database Migrations
Controlled changes to the database schema and data.

Application-Schema Compatibility
Application code must work with the current database schema.

Backward Compatibility
Both old and new application versions can run during deployment.

Expand and Contract Pattern
A safe way to make schema changes without breaking running applications.

Why It Happens

- Removing or renaming columns before all application instances are updated.
- Not testing migrations with the actual application versions.
- Assuming the application will be fully updated before the schema change affects it.
- Lack of rollback plan for database changes.
- Not considering dependencies such as stored procedures, views or other services.

Example: Removing a Column During a Rolling Deployment

Before Change

v1

Application v1 uses column email.

users
id
name
email

Schema Change

Column **email** is removed from the database.

users
id
name

After Deployment (Failure)

v1

Old application version still running tries to use **email** and fails.

Database error (column does not exist).

The Expand and Contract Migration Pattern (Recommended)

1. Add New Schema
Add new column or table (keep existing schema).

2. Deploy Compatible Application
Update application to work with both old and new schema.

3. Migrate Data
Backfill or migrate data to the new schema.

4. Verify
Confirm application works correctly and data is accurate.

5. Remove Old Schema Later
After all application versions are updated and verified, remove old columns or tables.

Prevention

- Plan schema changes and application releases together.
- Use backward compatible changes.
- Test migrations with real application versions.
- Separate destructive changes from application releases.
- Have a backup and recovery plan.

Important Considerations

- Test migrations in a staging environment that closely matches production.
- Validate application behavior with both old and new schema.
- Understand recovery options and limits.
- Check dependencies (stored procedures, views, other services).

Production Awareness

- Rolling back application code does not automatically reverse a database migration.
- Some schema changes (e.g. column removal) can permanently delete data if not planned carefully.
- Always verify compatibility before removing old structures.

Mistake 09: Deploying All Application Instances at Once

What Goes Wrong?

- A defective release affects every application instance before engineers can stop it.
- A small bug can cause a full outage if all instances are replaced at the same time.
- There is no safe window to detect and roll back.
- All users are impacted immediately.

Key Concepts

Rolling Deployment
Updates instances gradually, replacing a few at a time.

Canary Deployment
Releases to a small percentage of users or instances first.

Blue-Green Deployment
Runs two identical environments (blue and green) and switches traffic after verification.

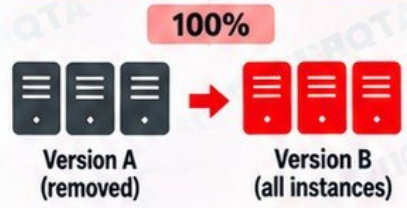
Traffic Shifting
Gradually increases traffic to the new version based on health and metrics.

Why It Happens

- Using a simple recreate deployment that replaces all instances at once.
- No progressive delivery strategy.
- Lack of health checks or stop conditions.
- Underestimating the risk of the release.
- Not considering application architecture, capacity and traffic patterns.

Deployment Strategies Comparison

All at Once (Not Recommended)



- All instances are replaced at once.
- If there is a problem, every user is affected.
- Higher risk of full outage.

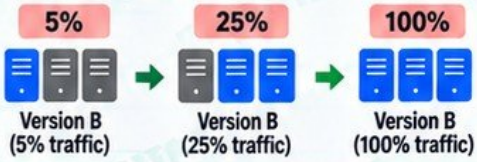
Rolling Deployment



- Gradual replacement of instances.
- Limits the blast radius.
- Allows detection and rollback if issues are found.

Canary and Blue-Green Deployment

Canary Deployment



Blue-Green Deployment

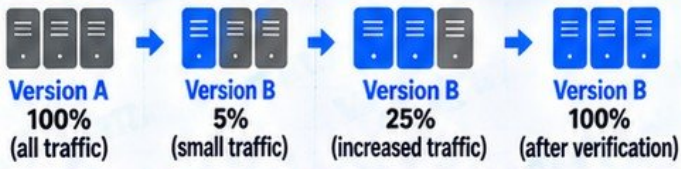


- Release to a small percentage first.
- Monitor metrics and user impact.
- Increase traffic gradually only after verification.

Example: Memory Leak Causes Full Outage

- A new release contains a memory leak.
- All instances are replaced at once.
- Memory usage increases and every instance starts failing under traffic.
- The application becomes unavailable for all users.

Progressive Release Flow (Example)



Prevention

- Use an appropriate progressive delivery strategy (rolling, canary or blue-green).
- Implement health checks, monitoring and automated or manual stop conditions.
- Choose a strategy based on your architecture, capacity and risk.
- Test the release in a staging environment before production.

Important Considerations

- Deployment strategy depends on architecture, traffic, capacity and risk.
- Set clear stop conditions (error rate, latency, resource usage).
- Have a rollback plan and test it regularly.

Mistake 10: Misconfiguring Health Checks and Readiness Probes

What Goes Wrong?

- A deployment appears successful even though the application cannot serve requests correctly.
- Pods are marked as healthy before the application is ready.
- Incorrect probe configuration causes unnecessary restarts or keeps unhealthy Pods in service.
- Users experience errors and failed requests.

Key Concepts

Startup Probe
Used for slow-starting applications to allow extra time to initialize before liveness and readiness probes start.

Readiness Probe
Checks if the application is ready to receive traffic. If it fails, the Pod is removed from Service endpoints and does not receive traffic.

Liveness Probe
Checks if the application is still running correctly. If it fails, Kubernetes restarts the container.

Important Differences

Running Container
A container can be running but the application inside is not ready to serve requests.

Ready for Traffic
A pod is ready when the readiness probe succeeds. Only ready pods receive traffic from the Service.

Liveness Failure
If the liveness probe fails, Kubernetes restarts the container which can cause unnecessary restarts if misconfigured.

How Probes Work in Kubernetes

Pod Starts
Container is created and the application starts.

→ **Startup Probe (optional)**
Gives the application time to initialize (used for slow starting applications).

→ **Readiness Probe**
When successful, pod is added to Service endpoints and receives traffic.

→ **Service Traffic**
Traffic is sent only to ready pods.

→ **Liveness Probe**
Continuously checks if the application is healthy. If it fails, Kubernetes restarts the container.

Example: Application Not Ready but Receiving Traffic

1. Application Starts
Application process starts but initialization is incomplete (e.g. loading data, connecting to services).

2. Misconfigured Probe
Readiness probe returns success too early, so the pod is marked as ready even though the application is not fully initialized.

3. Traffic Arrives (Failure)
Service sends traffic to the pod. Requests fail because required dependencies are not ready.

Example: Correct Readiness Probe

```yaml
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
  failureThreshold: 3
```

Correct vs Incorrect Configuration

Incorrect Readiness Probe

- Returns success before the application is actually ready.
- Uses a simple port check that does not verify application behavior.
- Causes traffic to be sent to unready pods.

Correct Readiness Probe

- Checks a meaningful application endpoint (e.g. /health/ready).
- Verifies required dependencies.
- Keeps pod out of service until fully ready.

Troubleshooting Commands

```
# Describe the pod and check probe events
kubectl describe pod <pod-name>

# View container logs for errors
kubectl logs <pod-name>
```

Prevention

- Configure startup, readiness and liveness probes based on real application behavior.
- Use appropriate thresholds (initialDelay, period, failureThreshold).
- Test probes in a staging environment before production.
- Verify that probes check the actual health and dependencies of the application.

Junior Engineer Tip

- Readiness controls traffic eligibility. Liveness determines whether Kubernetes should restart the container.
- A running container is not the same as a ready application.

Mistake 11: Running Deployment Pipelines With Excessive Privileges

What Goes Wrong?

- A compromised pipeline can change or delete resources far beyond its intended scope.
- A single mistaken command can cause widespread damage.
- Attackers can take control of high-privilege credentials and access all resources.
- Shared or long-lived credentials increase the risk of unauthorized access.
- Lack of separation of duties allows pipelines to perform actions they should not.

Key Concepts

Least Privilege
Give the minimum permissions required to perform specific tasks.

Separation of Duties
Different identities for build, test, and production deployment.

Kubernetes RBAC
Restrict what a service account can do and which resources it can access.

Cloud IAM
Use scoped roles instead of administrator-level access.

Protected Environments
Require additional approval, restricted access, and audit logs for production.

Why It Happens

- Using cluster-admin or administrator-level access for convenience.
- Giving the same credentials to multiple pipelines or teams.
- Using long-lived credentials instead of short-lived identities.
- Lack of proper access controls and environment isolation.
- Not reviewing permissions regularly.
- Believing the pipeline runs in a trusted and secure environment.

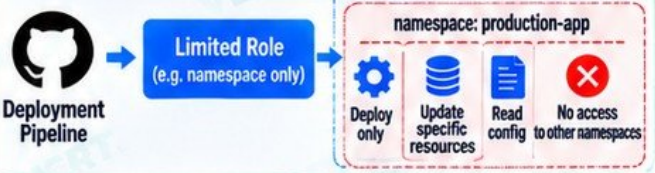
Example: Overprivileged Deployment Identity

Incorrect (Too Much Access)



The pipeline has permission to delete resources across the entire cluster, not just the intended namespace.

Correct (Scoped Access)



The pipeline can only perform the required actions in the specified namespace.

Example: Dangerous Command With High Privileges

```
# Accidental or malicious command
kubectl delete namespace --all
```

With excessive privileges, this command can delete all namespaces in the cluster.

Principle of Least Privilege



- Do not use cluster-admin for deployment pipelines.
- Avoid wildcard permissions (e.g. *).
- Do not share credentials across teams or projects.
- Grant only the permissions required for the specific task.
- Use separate identities for different environments.
- Review and audit permissions regularly.

Prevention

- Use least privilege and scoped deployment identities.
- Implement Kubernetes RBAC and cloud IAM with limited roles.
- Separate build, test and production environments.
- Use short-lived credentials and federated authentication where possible.
- Audit and monitor all privileged operations.
- Regularly review and remove unnecessary permissions.

Security Reminder

A deployment pipeline should not have more authority than it needs.

Mistake 12: Running Concurrent Deployments Without Coordination

What Goes Wrong?

- Two pipelines deploy to the same environment at the same time.
- Release order becomes unpredictable.
- Changes can overwrite each other or cause configuration conflicts.
- Partial deployments can leave the environment in an inconsistent state.
- Users may experience errors or downtime.

Key Concepts

Deployment Race
Two or more deployments modify the same resources at the same time, causing unpredictable results.

Shared-Environment Conflicts
Overlapping changes can conflict, making the final state unknown.

Deployment Serialization
Allow only one deployment to run at a time for a specific environment.

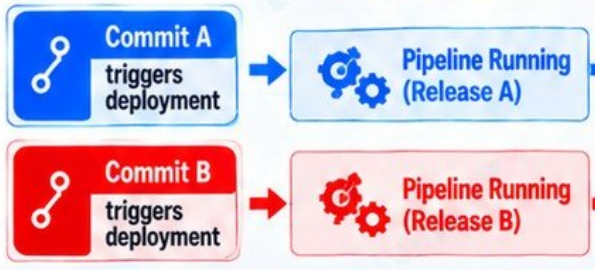
Concurrency Controls
Use environment locks, queues or pipeline settings to prevent overlapping deployments.

Cancellation Policies
Decide whether to cancel, queue or allow concurrent runs based on your deployment type and risk.

Why It Happens

- Multiple commits trigger deployments at the same time.
- No environment locking or concurrency control.
- Pipelines run independently without checking for existing deployments.
- Teams deploy to the same environment without coordination.
- Cancellation policies are misconfigured.
- Database migrations or infrastructure changes are not designed to run concurrently.

Example: Two Releases Deploying at the Same Time



Deploying to Production (same environment)

- Both pipelines modify the same resources.
- Changes can conflict.
- Final state is unpredictable.
- Users may see errors or downtime.

Possible Outcomes

- Release B overwrites changes from Release A.
- Partial deployment causes configuration drift.
- Database migration fails due to concurrent runs.
- Environment ends up in an inconsistent state.

How to Prevent It

- Allow only one deployment at a time for each environment.
- Use environment locks or concurrency controls in your CI/CD tool.
- Configure pipeline settings to queue or cancel overlapping runs.
- Coordinate database migrations and infrastructure changes.
- Make deployment operations safe to retry.
- Communicate deployment plans across teams.
- Monitor deployment activity and review logs.

Example: Concurrency Control (GitHub Actions)

```
concurrency:
  group: production-deployment
  cancel-in-progress: false
```

This ensures only one deployment to the production environment runs at a time. Additional runs wait in the queue.

Important Considerations

- Choose the right cancellation policy for your use case.
- Be careful when canceling deployments that include database migrations.
- Understand the impact on rollbacks and recovery.
- Test your deployment process, including queued and cancelled runs.

DevOps Connection

- CI jobs can run concurrently without issues.
- Production deployments change shared resources and often require coordination.
- Controlled and predictable releases reduce risk and make incidents easier to manage.

Mistake 13: Deploying Without Observability or Release Verification

What Goes Wrong?

- The pipeline reports success but the application experiences higher latency, errors or user-facing failures.
- Issues are not detected early enough.
- The application returns successful responses but produces incorrect results.
- Users are impacted before the problem is noticed.
- It becomes difficult to identify which deployment introduced the issue.

Key Concepts

Metrics
Quantitative data such as latency, requests, error rates, CPU and memory usage.

Logs
Detailed events that help you understand what happened.

Traces
Track a request as it moves through multiple services.

Deployment Events
Record when and what was deployed so you can correlate changes with production behavior.

Release Verification
Use automated and manual checks after deployment to confirm the application is working correctly.

Four Golden Signals

Latency
How long requests take to complete.

Traffic
The number of requests handled by the service.

Errors
The rate of failed requests (e.g. 4xx, 5xx).

Saturation
How close the service is to its resource limits (CPU, memory, disk, etc.).

Example: Deployment Appears Successful but Causes User Issues

1. Deploy New Version

Version v2 deployed

Pipeline reports success

2. Metrics After Deployment

Latency ↑
Errors ↑
Traffic —
Saturation ↑

3. User Impact

200 OK

Application returns HTTP 200 but produces incorrect business results.

4. Root Cause

Release v2 introduced a logic error that passes health checks but returns wrong data.

Release Annotations for Traceability

```yaml
metadata:
  annotations:
    deployment.veriqta.com/version: "v2.1.0"
    deployment.veriqta.com/commit: "a1b2c3d"
    deployment.veriqta.com/date: "2026-10-15"
    deployment.veriqta.com/actor: "ci-pipeline"
```

Post-Deployment Verification

- Run automated smoke tests.
- Check key application metrics.
- Verify critical user journeys.
- Monitor logs and error rates.
- Observe for a defined verification window (e.g. 5–30 minutes).
- Have a rollback plan if issues are detected.

Example: What to Monitor



Prevention

- Implement health checks and application metrics.
- Use logs and traces to detect issues quickly.
- Add release annotations for traceability.
- Run functional and business-level verification, not just technical checks.
- Monitor the application after deployment for a defined period.
- Set alerting for latency, error rate and saturation.

Important Considerations

- Monitor both technical and business metrics.
- Some issues do not appear immediately.
- Choose an appropriate verification window based on your application and risk.
- Ensure you can quickly roll back if needed.

Production Insight

A successful deployment must be evaluated against application behavior, not just pipeline status.

Mistake 14: Deploying Without a Tested Rollback or Recovery Plan

What Goes Wrong?

- Engineers discover a serious release defect but cannot restore service quickly or safely.
- A previous container image alone may not restore service after schema or configuration changes.
- Without a plan, recovery can be slow, risky, and may cause more downtime.
- Unclear steps lead to confusion, repeated mistakes, and longer outages.

Key Concepts

Rollback
Revert to a previous version of the application.

Roll-Forward
Fix and release a new version instead of reverting.

Recovery
Restore service using multiple methods (rollback, roll-forward, or manual steps).

Why Image Alone Isn't Enough
Schema or configuration changes may make the previous image incompatible.

Essential Tools
Deployment history, immutable artifacts, feature flags, and recovery runbooks.

Why It Happens

- No tested rollback plan or recovery runbook.
- Database or configuration changes break compatibility with older versions.
- Missing deployment history and artifact retention.
- Feature flags not used to control risky changes.
- Recovery procedures not tested before incidents.

Rollback, Roll-Forward and Recovery Options

Rollback



Return to a previous stable version.

Roll-Forward



Deploy a fixed version with the correct changes.

Recovery



Use multiple methods (rollback, roll-forward, manual recovery).

Required Elements

- Deployment history
- Immutable artifacts
- Feature flags
- Recovery runbooks

Example: Rollback Failure Due to Incompatible Changes

An application rollback fails because the previous version is incompatible with the updated database schema.

```
Error: column "user_status" does not exist
in table "users" (previous version)
```

Recovery Workflow



Prevention

- Maintain tested recovery procedures.
- Ensure migrations are compatible with older versions.
- Keep immutable artifacts and deployment history.
- Use feature flags for safer releases.
- Define clear incident decision criteria.

Junior Engineer Tip

A rollback plan is useful only when the team has verified that it can work.

Production Awareness

A previous container image alone may not restore service after schema or configuration changes.

Mistake 15: Treating Pipeline Success as the End of Deployment

What Goes Wrong?

- Engineers stop monitoring after the pipeline finishes.
- Failures appear later under real production traffic.
- Background jobs, scheduled tasks or data processes fail after deployment.
- Issues are not detected early, leading to user impact.
- The team assumes the release is successful because the pipeline reported success.

Key Concepts

Deployment vs Release
Deployment completion means the changes are in production. Release acceptance means the application is working correctly.

Post-Deployment Observation Period
Continue monitoring for a defined time after deployment (e.g. 30–60 minutes or longer).

Service-Level Indicators (SLIs)
Track key indicators such as latency, error rate, traffic, and job success rate.

Release Ownership
Clearly define who is responsible for monitoring and validating the release.

Incident Escalation
Have a clear escalation path if issues are detected after deployment.

Why It Happens

- The pipeline reports success, so the team assumes everything is working.
- Some failures take time to appear (traffic-dependent or background jobs).
- Monitoring and verification are not part of the release process.
- No clear ownership after deployment.
- Lack of defined acceptance criteria.
- Teams focus on deployment completion instead of application behavior.

Example: Pipeline Success but Failure Appears Later

1. Pipeline Completes

Deployment finished successfully.

Build	Success
Test	Success
Deploy	Success

2. Application Seems Fine

200 OK

Application responds correctly after deployment.

3. Issue Appears Later

30 minutes later, a background worker starts failing.

```
ERROR JobFailed
ERROR Processing failed for order #7842
```

4. User Impact

Orders are not processed, causing delays for users.

Users place orders → Orders not processed → Customer complaints

Prevention

- Define clear release acceptance criteria.
- Monitor key application metrics and workloads after deployment.
- Include background jobs and scheduled tasks in verification.
- Assign release ownership and handover.
- Set a post-deployment observation period.
- Document the final verification result.

Example SLIs to Monitor

- Latency (response time)
- Error rate (4xx, 5xx)
- Traffic (request count)
- Background job success rate
- Resource usage (CPU, memory)
- Business metrics (e.g. orders processed, payments completed)

Remember This

The release is not operationally complete until the required verification and handover steps are finished.

A Safe Deployment Pipeline

From Code Review to Verified Production Release

1 Pull Request + Review



- Review code changes
- Check for risks
- Ensure required approvals
- Verify linked work items

2 Tests + Security Checks



- Run automated tests (unit, integration, end-to-end)
- Perform security scans (SAST, DAST, dependency checks)
- Check for policy violations
- Fail if critical issues are found

3 Build + Verify Artifact



- Build a traceable, immutable artifact
- Include version, commit, and metadata
- Verify the artifact (e.g. checksum, signature)
- Store in a secure artifact registry

4 Controlled Production Deployment



- Promote the verified artifact without rebuilding it
- Deploy using restricted permissions
- Use a controlled release strategy (rolling, canary, blue-green)
- Apply infrastructure and configuration controls

5 Smoke Tests + Monitoring



- Run post-deployment smoke tests
- Verify key functional flows
- Monitor metrics, logs and traces
- Check application and business indicators (SLIs)
- Observe for a defined period (e.g. 30–60 minutes)

6 Release Decision

- Application Healthy
- Accept Release (Complete handover)
- Application Unhealtty
- Stop + Recover (Rollback or roll-forward)

Key Principles

- Use a single, verified artifact.
- Do not rebuild the artifact for production.
- Apply least privilege and environment controls.
- Test and monitor real application behavior.
- Keep recovery options and a clear rollback plan.
- Define release acceptance criteria.
- Maintain traceability from code to production.
- Establish ownership and handover.

What a Verified Deployment Achieves

- Validates changes before production.
- Reduces configuration and environment drift.
- Detects security and functional issues early.
- Minimizes the impact of failed releases.
- Provides visibility into application health.
- Ensures quick recovery when needed.
- Builds a reliable and repeatable release process.

Important Reminders

- Pipeline success is not the same as release success.
- Some issues appear only under real production traffic.
- Background jobs and scheduled tasks must also be verified.
- A rollback plan is useful only when it has been tested.
- The release is not complete until verification and handover steps are finished.

Practice: Which Pipeline Stage Prevents Each of the 15 Mistakes?

#	Mistake	Pipeline Stage(s) That Help Prevent It
1	Deploying without testing	Stage 2 – Tests + Security Checks
2	Skipping security checks	Stage 2 – Tests + Security Checks
3	Using mutable or unversioned images	Stage 3 – Build + Verify Artifact
4	Deploying without a clear strategy	Stage 4 – Controlled Production Deployment
5	Not validating configuration changes	Stage 2 and Stage 4 – Tests and Controlled Deployment
6	Skipping dependency checks	Stage 2 – Tests + Security Checks
7	Allowing development and production to drift	Stage 2 and Stage 4 – Tests and Controlled Deployment
8	Deploying database changes without compatibility planning	Stage 2 and Stage 4 – Tests and Controlled Deployment
9	Deploying all application instances at once	Stage 4 – Controlled Production Deployment
10	Misconfiguring health checks and readiness probes	Stage 4 and Stage 5 – Deployment and Monitoring
11	Running pipelines with excessive privileges	Stage 4 – Controlled Production Deployment
12	Running concurrent deployments without coordination	Stage 4 – Controlled Production Deployment
13	Deploying without observability or release verification	Stage 5 – Smoke Tests + Monitoring
14	Deploying without a tested rollback or recovery plan	Stage 4 and Stage 6 – Deployment and Release Decision
15	Treating pipeline success as the end of deployment	Stage 5 and Stage 6 – Monitoring and Release Decision

@veriqta

VERIQTA

Troubleshooting Lab:
A Green Pipeline, a Broken Application

Incident Scenario

A Kubernetes deployment reports success.
Five minutes later, users experience HTTP 503 errors.

- The pipeline shows green and the deployment completed.
- New Pods are running.
- Users cannot access the application (HTTP 503).
- You must investigate using evidence, not assumptions.

What You Need to Find

1. Did the correct container image deploy?
2. Are the new Pods running and ready?
3. Are readiness probes failing?
4. Does the Service have ready endpoints?
5. Are application logs reporting connection or startup failures?
6. Did error rates increase immediately after the release?

Investigation Flow

Deployment reported success (pipeline is green)

↓

Users get HTTP 503 errors

↓

Investigate using commands and evidence

↓

Identify the root cause

↓

Decide: Investigate further, pause rollout or recover

Diagnostic Commands

These commands assume the correct cluster context and namespace.
The EndpointSlice listing must be inspected to identify the slices associated with the relevant Service.

1 Check rollout status

```
kubectl rollout status
 deployment/web
```

Verify that the deployment completed successfully.

2 Check Pods

```
kubectl get pods
```

Confirm the new Pods are running and check their readiness status (READY column).

3 Describe deployment

```
kubectl describe
 deployment web
```

Verify the deployed image, replica count, events and any probe or scheduling issues.

4 Check Service endpoints

```
kubectl get endpointslices
```

Look for the EndpointSlices related to your Service and confirm that it has ready endpoints.

5 Check application logs

```
kubectl logs deployment/web
--tail=50
```

Look for error messages, connection failures, or startup issues.

6 (Optional) Check recent events

```
kubectl get events
--sort-by=.lastTimestamp
```

Review recent events for warnings or failures related to the deployment, Pods, or Service.

Exercise

Use the evidence to decide the next action:

1. Determine whether the correct image deployed.
2. Identify the most likely cause of the 503 errors.
3. Decide to investigate further, pause the rollout, or initiate recovery.
4. Document your findings and decision.

Verification

- Pods are running and ready.
- Service has healthy endpoints.
- Readiness probes are succeeding.
- Error rates have reduced.
- User-facing requests return successful responses (e.g. HTTP 200).

Key Lesson

A successful deployment in the pipeline does not mean the application is working correctly. Always verify production behavior using real evidence.

The Production Deployment Readiness Checklist

Main Teaching Focus:
Give junior engineers a practical checklist they can use before, during, and after a release.



1 Before Deployment

- [x] Required tests and reviews passed.
- [x] Security checks and artifact verification completed.
- [x] Configuration and database compatibility confirmed.
- [x] Deployment permissions and release strategy reviewed.
- [x] Recovery procedure and responsible engineer identified.

2 During Deployment

- [x] Correct immutable artifact selected.
- [x] Deployment changes controlled and traceable.
- [x] Readiness and rollout progress checked.
- [x] Error rates, latency, and application behavior observed.

3 After Deployment

- [x] Smoke tests and critical user journeys verified.
- [x] Logs, metrics, and alerts reviewed.
- [x] Recovery initiated if acceptance criteria fail.
- [x] Release outcome and evidence recorded.
- [x] Operational ownership confirmed.

Final Engineering Principle:

A production-ready pipeline does more than deploy code. It helps engineers prevent unsafe releases, detect failures, limit impact, and recover safely.